Design Patterns

C# Developer Reference — Session 2

Singleton · Factory · Observer

with Exercises & Solutions

Quick Reference

Pattern	Category	Problem	Key idea
Singleton	Creational	Need exactly one instance app-wide	Private constructor + Lazy instance
Factory	Creational	Object creation scattered everywhere	One place creates all objects
Observer	Behavioral	Notify many objects on state change	Subscribe / Unsubscribe / Notify

1. Singleton Pattern

"Ensure a class has only one instance throughout the entire application."

Three rules:

- Private constructor — blocks new ClassName() from outside
- Static Lazy instance — lives at class level, created only once
- Public static property — the only way to get the instance

■ Without Singleton

```
// ■ BAD — new instance every time, no control

var db1 = new DatabaseConnection();

var db2 = new DatabaseConnection();

// db1 and db2 are different objects — wasteful
```

■ Singleton with Lazy

```
public class DatabaseConnection

{

    // Lazy<T> — does not create the object until first use (.Value)

    // static  — belongs to the class, not to any instance

    // readonly — cannot be reassigned after declaration

    private static readonly Lazy<DatabaseConnection> _instance =

        new Lazy<DatabaseConnection>(() => new DatabaseConnection());


    // Private constructor — empty body is fine.

    // Its only job: prevent anyone calling new DatabaseConnection()

    private DatabaseConnection()

    {
```

```
        Console.WriteLine("DB connection created — only once!");
    }

    // The one and only entry point from outside
    // .Value → first call creates the object; later calls return same object
    public static DatabaseConnection Instance => _instance.Value;

    public void Query(string sql) => Console.WriteLine($"Running: {sql}");
}

// Usage
DatabaseConnection.Instance.Query("SELECT * FROM Users");
// DB connection created — only once!
// Running: SELECT * FROM Users

DatabaseConnection.Instance.Query("SELECT * FROM Orders");
// Running: SELECT * FROM Orders   ← constructor did NOT run again
```

■ *Lazy is a wrapper. Think of it as a box — empty at first. The moment you open it (.Value) for the first time, the object is created and stored inside. Every open after that returns the same object already in the box.*

2. Factory Pattern

> *"Centralise object creation — one place creates all objects of a family."*

When new Dog(), new Cat() are scattered across many files, adding a new type means hunting through all of them. Factory brings all creation logic to one place.

■ Without Factory

```
// ■ BAD — creation logic scattered, every file must change for new types
public class AnimalShelter
{
    public void Adopt(string type)
    {
        if (type == "Dog")       { new Dog().MakeSound(); }
        else if (type == "Cat") { new Cat().MakeSound(); }
        // Adding Bird? Must find and edit every file like this
    }
}
```

■ Factory with Dictionary — Open/Closed

```
public interface IAnimal { void MakeSound(); }
public class Dog  : IAnimal { public void MakeSound() => Console.WriteLine("Woof!"); }
public class Cat  : IAnimal { public void MakeSound() => Console.WriteLine("Meow!"); }
public class Bird : IAnimal { public void MakeSound() => Console.WriteLine("Tweet!"); }

// ■■ Advanced Factory — fully Open/Closed ■■■■■■■■■■■■■■■■■■■■■■■■■■
public class AnimalFactory
```

```
{
    // Key   = animal name
    // Value = Func<IAnimal> — a function (recipe) that creates the animal
    //         stored but NOT called yet — called only when Create() runs
    private static readonly Dictionary<string, Func<IAnimal>> _registry = new()
    {
        { "Dog",  () => new Dog()  },
        { "Cat",  () => new Cat()  },
        { "Bird", () => new Bird() },
    };

    // Register new animal from outside — never touch Create()
    public static void Register(string type, Func<IAnimal> creator)
        => _registry[type] = creator;

    public static IAnimal Create(string type)
    {
        // TryGetValue — safe lookup; returns false instead of crashing
        // out var creator — Dictionary fills this variable if key is found
        if (_registry.TryGetValue(type, out var creator))
            return creator();   // call the stored function → create object
        throw new ArgumentException($"Unknown animal: {type}");
    }
```

```
}


// Usage

AnimalFactory.Create("Dog").MakeSound();   // Woof!


// Add Fish with zero changes to AnimalFactory

AnimalFactory.Register("Fish", () => new Fish());

AnimalFactory.Create("Fish").MakeSound();
```

■ *Func is the type of a function that takes no parameters and returns IAnimal. The lambda () => new Dog() is that function — stored in the Dictionary but not yet called. creator() actually calls it and creates the object.*

3. Observer Pattern

"When one object changes, all objects watching it are notified automatically."

Two roles:

- Subject (StockMarket) — manages the observer list, calls Notify on change
- Observer (InvestorObserver, BotObserver) — reacts when notified

■ Without Observer

```
// ■ BAD — subject hardwired to every concrete notifier

public class StockMarket

{

    public void PriceChange(decimal price)

    {

        new EmailNotifier().Notify(price);   // tightly coupled

        new SmsNotifier().Notify(price);     // adding BotNotifier? edit here

    }

}
```

■ Observer — loosely coupled

```
// Contract for all observers

public interface IObserver { void Update(decimal price); }


// Concrete observers — each handles the notification differently

public class InvestorObserver : IObserver

{

    private readonly string _name;
```

```
    public InvestorObserver(string name) => _name = name;

    public void Update(decimal price)

        => Console.WriteLine($"Investor {_name}: price changed to {price}");

}


public class BotObserver : IObserver

{

    public void Update(decimal price)

        => Console.WriteLine($"Bot triggered: auto-sell at {price}");

}


// Subject — only knows IObserver, never the concrete types

public class StockMarket

{

    private decimal _currentPrice;

    private readonly List<IObserver> _observers = new();


    public void Subscribe(IObserver o)   => _observers.Add(o);

    public void Unsubscribe(IObserver o) => _observers.Remove(o);


    public void PriceChange(decimal newPrice)

    {

        if (newPrice == _currentPrice) return;   // no change, no notify

        _currentPrice = newPrice;
```

```
        foreach (var o in _observers) o.Update(newPrice);
    }
}

// Usage
var market = new StockMarket();
var investor = new InvestorObserver("Trung");
var bot      = new BotObserver();
market.Subscribe(investor);
market.Subscribe(bot);
market.PriceChange(150m);
// Investor Trung: price changed to 150
// Bot triggered: auto-sell at 150
market.Unsubscribe(bot);
market.PriceChange(200m);
// Investor Trung: price changed to 200   ← bot receives nothing
```

■ *Observer connects directly to SOLID: D — StockMarket depends on IObserver (abstraction), not InvestorObserver or BotObserver (concrete). O — Add BotObserver or any new observer type without touching StockMarket.*

Exercises

═■ Exercise 1 — Find the violations (SOLID)

Read the code below. Identify which SOLID principles are violated and why.

Do not fix — just analyse.

```
public class OrderService

{

    public void PlaceOrder(string productName, int quantity, string userEmail)

    {

        if (quantity <= 0)

            throw new Exception("Quantity must be greater than 0");


        var conn = new SqlConnection("Server=...;Database=...");

        conn.Open();

        var cmd = new SqlCommand("INSERT INTO Orders VALUES (@p,@q)", conn);

        cmd.Parameters.AddWithValue("@p", productName);

        cmd.Parameters.AddWithValue("@q", quantity);

        cmd.ExecuteNonQuery();


        if (productName == "Laptop")

            Console.WriteLine($"Shipping Laptop to {userEmail}");

        else if (productName == "Phone")

            Console.WriteLine($"Shipping Phone to {userEmail}");
```

```
        var smtp = new SmtpClient("smtp.gmail.com");

        smtp.Send("no-reply@shop.com", userEmail, "Order confirmed!", "...");

    }

}
```

═■ Exercise 2 — Refactor (SOLID)

Refactor the code above to follow SOLID principles.

Split into: OrderValidator, OrderRepository, ShippingService (with interface), NotificationService (with interface), OrderService.

═■ Exercise 3 — Singleton

Write an AppConfig Singleton containing:

• AppName = "MyApp" • Version = "1.0.0" • MaxUsers = 100

Use Lazy. In Main, call AppConfig.Instance twice and prove it is the same object.

═■ Exercise 4 — Factory

Write a NotificationFactory that creates:

• EmailNotification — prints "Sending email: {message}"

• SmsNotification — prints "Sending SMS: {message}"

• PushNotification — prints "Sending push: {message}"

Use Dictionary + Register() so new types can be added without editing the factory.

═■ Exercise 5 — Observer

Write a StockMarket (subject). When price changes, notify:

• InvestorObserver — prints "Investor {name}: price changed to {price}"

• BotObserver — prints "Bot triggered: auto-sell at {price}"

> Only notify when the price actually changes (skip duplicates).

Solutions

Exercise 1 — Violations

Principle	Violation
S — Single Responsibility	PlaceOrder does 4 jobs: validate, save to DB, ship, send email. Each job is a separate reason to change.
O — Open/Closed	Adding a new product (Tablet) requires editing PlaceOrder. Should extend without modifying existing code.
D — Dependency Inversion	PlaceOrder directly creates SmtpClient (concrete). Should depend on INotificationService (abstraction).

Exercise 2 — Refactored OrderService

■ **Solution**

```
public interface IOrderValidator   { void Validate(string product, int qty); }

public interface IOrderRepository  { void Save(string product, int qty, string email);
}

public interface IShippingStrategy { void Ship(string product, int qty, string email);
}

public interface INotificationService { void Notify(string email, string product); }


public class OrderValidator : IOrderValidator

{

    public void Validate(string product, int qty)

    {

        if (qty <= 0) throw new ArgumentException("Quantity must be > 0");

    }

}
```

```
public class OrderRepository : IOrderRepository

{

    public void Save(string product, int qty, string email)

        => Console.WriteLine($"Saved: {qty} x {product} for {email}");

}


public class LaptopShipping : IShippingStrategy

{

    public void Ship(string product, int qty, string email)

        => Console.WriteLine($"Special shipping {qty} x {product} to {email}");

}


public class EmailNotification : INotificationService

{

    public void Notify(string email, string product)

        => Console.WriteLine($"Email to {email}: order for {product} confirmed");

}


public class OrderService

{

    private readonly IOrderValidator _v;

    private readonly IOrderRepository _r;

    private readonly IShippingStrategy _s;
```

```
    private readonly INotificationService _n;

    public OrderService(IOrderValidator v, IOrderRepository r,
                        IShippingStrategy s, INotificationService n)
        => (_v, _r, _s, _n) = (v, r, s, n);

    public void PlaceOrder(string product, int qty, string email)
    {
        _v.Validate(product, qty);
        _r.Save(product, qty, email);
        _s.Ship(product, qty, email);
        _n.Notify(email, product);
    }
}
```

Exercise 3 — AppConfig Singleton

■ Solution

```
public class AppConfig
{
    private static readonly Lazy<AppConfig> _instance =
        new Lazy<AppConfig>(() => new AppConfig());

    public static AppConfig Instance => _instance.Value;

    public string AppName  { get; private set; }
```

```
    public string Version  { get; private set; }
    public int    MaxUsers { get; private set; }

    private AppConfig()
    {
        AppName  = "MyApp";
        Version  = "1.0.0";
        MaxUsers = 100;
        Console.WriteLine("AppConfig created — only once!");
    }
}

// Main
var c1 = AppConfig.Instance;
var c2 = AppConfig.Instance;
Console.WriteLine(ReferenceEquals(c1, c2));  // True
Console.WriteLine(c1.AppName);               // MyApp
```

Exercise 4 — NotificationFactory

■ Solution

```
public interface INotification { void Send(string message); }

public class EmailNotification : INotification
    { public void Send(string msg) => Console.WriteLine($"Sending email: {msg}"); }
```

```
public class SmsNotification : INotification

    { public void Send(string msg) => Console.WriteLine($"Sending SMS: {msg}"); }

public class PushNotification : INotification

    { public void Send(string msg) => Console.WriteLine($"Sending push: {msg}"); }


public class NotificationFactory

{

    private static readonly Dictionary<string, Func<INotification>> _registry = new()

    {

        { "email", () => new EmailNotification() },

        { "sms",   () => new SmsNotification()   },

        { "push",  () => new PushNotification()  },

    };


    public static void Register(string type, Func<INotification> creator)

        => _registry[type] = creator;


    public static INotification Create(string type)

    {

        if (_registry.TryGetValue(type.ToLower(), out var creator))

            return creator();

        throw new ArgumentException($"Unknown type: {type}");

    }
```

```
}


// Usage
NotificationFactory.Create("email").Send("Hello!");  // Sending email: Hello!
NotificationFactory.Register("slack", () => new SlackNotification());
```

Exercise 5 — StockMarket Observer

■ Solution

```
public interface IObserver { void Update(decimal price); }


public class InvestorObserver : IObserver
{
    private readonly string _name;
    public InvestorObserver(string name) => _name = name;
    public void Update(decimal price)
        => Console.WriteLine($"Investor {_name}: price changed to {price}");
}


public class BotObserver : IObserver
{
    public void Update(decimal price)
        => Console.WriteLine($"Bot triggered: auto-sell at {price}");
}


public class StockMarket
```

```
{
    private decimal _currentPrice;
    private readonly List<IObserver> _observers = new();

    public void Subscribe(IObserver o)   => _observers.Add(o);
    public void Unsubscribe(IObserver o) => _observers.Remove(o);

    public void PriceChange(decimal newPrice)
    {
        if (newPrice == _currentPrice) return;
        _currentPrice = newPrice;
        foreach (var o in _observers) o.Update(newPrice);
    }
}

// Main
var market   = new StockMarket();
var investor = new InvestorObserver("Trung");
var bot      = new BotObserver();
market.Subscribe(investor);
market.Subscribe(bot);
market.PriceChange(150m);
// Investor Trung: price changed to 150
```

```
// Bot triggered: auto-sell at 150

market.Unsubscribe(bot);

market.PriceChange(200m);

// Investor Trung: price changed to 200
```

■ *All three patterns connect back to SOLID. Singleton enforces S (one responsibility: manage the single instance). Factory and Observer both apply O (extend without modifying) and D (depend on abstractions, not concrete types). Patterns are not separate rules — they are SOLID in action.*